

Aufgabenblatt 1

Projektskizze: Schwarmintelligenz

Agententechnologien — Grundlagen und Anwendungen
TU Berlin, Sommersemester 2026

Gruppe 11

Ivan Wanylyn

Abgabe: 04.05.2026

Aufgabenblatt 1 - Projektskizze: Schwarmintelligenz

Bearbeiter: Gruppe 11

1. Aufgabenstellung in Kurzform

Implementiert wird eine reaktive Multi-Agenten-Simulation einer Ameisenkolonie, die in einer 2D-Gitterwelt Futter sammelt. Die Ameisen orientieren sich ausschließlich lokal über Pheromonspuren (Nest- und Futterpheromon). Ziel der Projektskizze ist der Systementwurf: Klassendiagramm, Sequenzdiagramm, Datenmodell der Startkonfiguration, Logging-Plan sowie eine Forschungsfrage. Die Implementierung folgt in der Projektdokumentation.

2. Forschungsfrage

2.1 Leitfrage

Wie verhält sich eine Ameisenkolonie, wenn Nahrung in unterschiedlicher Entfernung vom Nest liegt? Werden alle Quellen gleich gut erreicht oder bevorzugt die Kolonie die nähere?

2.2 Vermutung

Wir erwarten, dass die nähere Quelle zuerst gefunden und stabil ausgebeutet wird, bevor sich Trails zur weiter entfernten Quelle etablieren — schlicht weil kürzere Trails weniger durch Pheromonverdunstung verlieren.

2.3 Was wir messen

Pro Lauf protokollieren wir, wann das erste Nahrungsitem aus jeder Quelle im Nest ankommt ($t_{\text{first}}(A)$, $t_{\text{first}}(B)$) und wie viel Nahrung im Zeitverlauf eingelagert wird. Wir variieren die Distanz der zweiten Quelle ($D_2 \in \{12, 15, 20\}$ gegenüber $D_1 = 5$) und die Ameisenzahl ($N \in \{10, 20, 30\}$).

2.4 Nebenfrage

Im Warmstart-Experiment schauen wir zusätzlich, wie schnell sich ein Trail erholt, wenn die Pheromone für kurze Zeit ausgeschaltet werden — erwartet ist, dass die Kolonie relativ zügig zurückfindet.

3. Domänen- und Systemmodell

3.1 Begriffe

Begriff	Bedeutung
GridWorld	$n \times m$ -Raster aus <code>Cell</code> -Objekten; toroidal <i>aus</i> (Ränder blockieren).
Cell	Feld mit Kapazität $c \geq 0$ (0 = Hindernis), Item-Stack, Pheromon-Levels.
Agent	Ameise mit Position, Energie, Tragzustand, Bewegungsregeln.
Manager	zentrale Instanz: Zeitsteuerung, Wahrnehmung, Konfliktauflösung, Logging.
Tick	Diskreter Zeitschritt; jeder Agent führt genau eine Aktion aus.
Item	Auf Feld liegendes Objekt: <code>Food</code> , <code>EnergySource</code> , <code>NestMarker</code> .
Pheromon	Skalares Feldattribut (<code>nest</code> , <code>food</code>) mit Verdunstungsrate p .

3.2 Klassendiagramm

Siehe [diagrams/class-diagram.mmd](#) (Struktur) und [diagrams/class-diagram-types.mmd](#) (Wertobjekte). Eingebunden, kommentiert in Abschnitt 4.

3.3 Sequenzdiagramm

Siehe [diagrams/sequence-diagram.mmd](#). Beschreibung des Tick-Ablaufs in Abschnitt 5.

3.4 Konfigurationsmodell

Beispiel: [config/world_default.json](#) + Schema [config/world.schema.json](#). Erläuterung in Abschnitt 6.

4. Klassendiagramm - Erläuterung

Abbildung 1a: Klassendiagramm – Struktur und Komposition.

Abbildung 1b: Klassendiagramm – Wertobjekte (Item-/Action-Hierarchie) und Botschaftsobjekte zwischen Manager und Agent.

Das Klassendiagramm ist auf zwei Panels aufgeteilt, damit jedes Panel auf einer A4-Seite lesbar bleibt: **(a)** zeigt die strukturelle Komposition (Manager, Welt, Konfiguration,

Logging, RNG), **(b)** zeigt die Wertobjekte für die Kommunikation (Perception, ActionRequest) sowie die Vererbungshierarchien für Item und Action.

4.1 Verantwortlichkeiten (Kern)

PDF p.3 fordert für die Skizze die Erläuterung von **Gridwelt, Agenten und Manager** sowie der **Methoden für den Nachrichtenaustausch** (siehe § 4.2).

- **SimulationManager** – Kontrolliert Zeit, Welt und Agentenzustände (PDF Hinweise §"Manager"): startet/terminiert die Simulation, stellt Wahrnehmungen zu, führt Aktionen aus, prüft Energielevel, protokolliert. Hat als einziger vollständiges Weltwissen.
- **GridWorld** – $n \times m$ -Matrix von Cell. `capacity == 0` $\Rightarrow$ Hindernis. Items und Pheromone liegen auf den Feldern.
- **AntAgent** – Reaktiver Agent. Pro Tick `Sense` $\rightarrow$ `Reason` $\rightarrow$ `Act` (PDF Hinweise §"Agenten"). Interner Zustand: `position`, `energy`, `carrying`, optional Kurzzeitgedächtnis (letzte Richtung) zur Zyklusvermeidung.

Weitere Hilfsklassen (Werteobjekte für Aktionen, Wahrnehmungen, Logger etc.) sind im Klassendiagramm zu sehen.

4.2 Botschafts-/Schnittstellenmodell

Manager und Agent kommunizieren über zwei Werteobjekte: der Manager liefert eine **Wahrnehmung** (was der Agent gerade sieht — Items am aktuellen Feld und Pheromone der Nachbarschaft), der Agent gibt eine **Aktion** zurück (`Move`/`PickUp`/`Drop`/`NoOp`). Eine asynchrone Message-Queue brauchen wir nicht, da die Simulation single-threaded läuft (vgl. PDF Hinweise: „kein Multithreading“).

4.3 Konfliktauflösung (zentral im Manager)

Konflikt: mehrere Agenten wollen in derselben Tick-Runde auf dasselbe Feld, dessen Kapazität nicht reicht. Strategie laut PDF Hinweise §"Manager": **first come, first serve** über die Reihenfolge der Agenten-Indizes — spätere Anwärter erhalten eine Fehler rückmeldung und führen implizit `NoOp` aus. Die fixe Reihenfolge ist zwar nicht fair, aber laut PDF akzeptabel und hält die Simulation deterministisch.

5. Sequenzdiagramm – Tick-Ablauf

Abbildung 2: Sequenzdiagramm eines Ticks.

Pro Tick fährt der `SimulationManager` die in den PDF Hinweisen §"Manager" / §"Agenten" beschriebene Schleife. Pro Agent gilt **Sense** $\rightarrow$ **Reason** $\rightarrow$ **Act**; der Manager rahmt das mit Aktionsausführung und Pheromon-Verdunstung ein.

#	Phase	Akteur	Beschreibung
1	apply_actions	Manager	Verarbeitet die aus dem Vortick gesammelten Actions <i>first-come-first-served</i> . Zieht Energie ab, schreibt Erfolg/Misserfolg in die Eingabequeue der Agents.
2	sense	Manager → Agent	Stellt jedem Agenten Wahrnehmungen zu: Items am aktuellen Feld und in der 4er-Nachbarschaft (PDF p.2).
3	reason → act	Agent	Agent entscheidet probabilistisch-reaktiv und legt seine nächste Aktion fest. Meldet Entscheidung an den Manager.
4	update_pheromones	Manager	evaporate(p) für alle Felder.
5	log_and_advance	Manager / Logger	Tick-Metriken schreiben, tick++.

6. Konfigurations-/Modelldatei

6.1 Anforderungen

- **Reproduzierbarkeit:** RNG-Seed im File.
- **Vollständigkeit:** Topologie, Initialpositionen, Energie-/Aktionskosten, Pheromonparameter, Abbruchkriterien, Logging-Optionen.
- **Validierbarkeit:** Ein JSON-Schema liegt bei und wird beim Start geprüft.

6.2 Format

Siehe `config/world_default.json` (Beispiel) und `config/world.schema.json` (formales Schema). Die wichtigsten Top-Level-Felder:

```
{
  "schema_version": "1.0.0",
  "seed": 20260427,
  "grid": { "width": 30, "height": 20, "default_capacity": 4 },
  "obstacles": [
    { "x": 10, "y": 5, "w": 1, "h": 8 },
    { "x": 18, "y": 12, "w": 4, "h": 1 }
  ],
  "nests": [{ "id": 0, "x": 2, "y": 10, "capacity": 9999 }],
  "food": [
    { "x": 27, "y": 3, "amount": 200 },
    { "x": 25, "y": 17, "amount": 150 }
  ],
  "agents": { "count": 50, "spawn": "nest", "initial_energy": 500, "energy_max": 1000 },
  "costs": { "move": 1, "pickup": 0, "drop": 0, "noop": 0, "perceive": 0 },
  "pheromones": { "evaporation": 0.02, "deposit_initial": 100.0, "decay_per_step": 0.9, "min_level": 0.01 },
  "termination": { "max_ticks": 5000, "stop_when_food_zero": true },
  "logging": { "level": "info", "metrics_every": 10, "trace_agents": [0, 1], "output_dir": "logs" }
}
```

7. Logging-Plan

7.1 Was wir loggen

- Eine **JSONL-Datei pro Lauf** (`runs/<run_id>/events.jsonl`): pro Tick eine Zeile mit den Aggregatmetriken (Futter im Nest, mittlere Energie, aktive/tragende Ameisen, Pheromonsummen, Konflikte).
- **Konsolen-Summary** alle `metrics_every` Ticks — eine kompakte Zeile, damit man beim Debuggen sieht, was passiert.
- **Optional pro Agent** ein Trace mit Position, Aktion, Energie — nur für die Agenten, die in der Konfiguration gelistet sind.

7.2 Aggregation und Auswertung

Die `events.jsonl` aller Läufe werden zu einer `summary.csv` zusammen gefasst (eine Zeile pro Lauf mit Seed und den wichtigsten Kennzahlen). Für die Auswertung planen wir Plots mit matplotlib: Sammelrate über Zeit, Vergleich der `t_first`-Werte je Quelle, Pheromon-Heatmap-Snapshots. Die konkreten Auswertungsskripte schreiben wir in der Implementierungs phase.

7.3 Score (optional)

Falls für Vergleiche zwischen Konfigurationen nützlich, kann eine einfache Linearkombination wie $\alpha \cdot \text{food_in_nest} - \beta \cdot \text{total_energy_spent}$ gebildet werden. Wir entscheiden im Lauf der Implementierung, ob wir das wirklich brauchen — die Rohmetriken aus § 7.1 reichen für die geplanten Experimente.

8. Geplante Experimente

Aufgabenblatt p.2 verlangt genau **drei** Experimente mit je 2–3 Simulationen. Zuordnung zur dortigen Nummerierung:

#	Name (PDF)	Zweck	Varierte Parameter	Erwartung
1	<i>Funktion + Distanz</i>	PDF-Exp 1: Sammeln funktioniert + prüft die Vermutung.	$N \in \{10, 20, 30\}$; zwei Quellen $D_1 = 5$, $D_2 \in \{12, 15, 20\}$; 2–3 Seeds.	<code>food_in_nest</code> > 0 ; nähere Quelle wird sichtbar früher ausgebeutet als die ferne.
2	<i>Warm-Start</i>	PDF-Exp 2: Trails	Pheromone in Tick 1500 für 50 Ticks ausschalten.	Sammelrate erholt sich

#	Name (PDF)	Zweck	Variierte Parameter	Erwartung
		erholen sich nach Bruch.		auf $\geq 90\%$ in < 500 Ticks.
3	Skalierung	PDF-Exp 3: Chancen/Probleme der Skalierung.	Eine Achse: Quellenabstand, Anzahl Quellen, oder Verdunstungsrate ρ .	Sichtbare Trade-offs (z. B. höheres ρ bricht ferne Trails ab).

Jede Experimentkonfiguration wird als eigene `experiment_*.json` neben `world_default.json` abgelegt, damit Läufe per `--config` reproduzierbar sind. Eine Brokering-Variante (PDF Skript Kap. 6.3) folgt in der Projektdokumentation und ist nicht Teil dieser Skizze.

9. Designentscheidungen

Wir entwerfen das System **single-threaded und deterministisch** — das Aufgabenblatt rät von Parallelität ab, und das Debugging wird so deutlich einfacher. Der **Manager hält die einzige Sicht auf die Welt**; Agenten greifen ausschließlich über die vom Manager gestellten Wahrnehmungen darauf zu, damit sie die Welt nicht versehentlich mutieren. Die **Konfiguration** lesen wir aus **JSON** (deklarativ, gut prüfbar gegen ein Schema). **Pheromone** deponieren wir mit fallender Stärke (decay_per_step^k pro Schritt), damit Trails zur Quelle hin stärker werden. Implementiert wird in **Python**: schneller Prototyp-Zyklus, gute Bibliotheken (numpy, matplotlib, JSON-Schema-Validatoren), und das Tick-Modell bleibt sprach-agnostisch.

10. Risiken und offene Punkte

Risiko / Frage	Mitigation
Ameisen verirren sich, bevor Trails entstehen.	Anfangsbias + längere Erkundungsphase.
Vermutung trivial, falls eine Quelle vor Trail etablierung erschöpft.	Hinreichend Nahrung pro Quelle; mehrere (N, D_2) -Kombinationen.

11. Anhänge

- `diagrams/class-diagram.mmd` – Mermaid-Klassendiagramm (Struktur).

- `diagrams/class-diagram-types.mmd` – Mermaid-Klassendiagramm (Wertobjekte / Botschaftsobjekte).
- `diagrams/sequence-diagram.mmd` – Mermaid-Sequenzdiagramm.
- `config/world_default.json` – Kommentierte Beispielkonfiguration.
- `config/world.schema.json` – JSON-Schema zur Validierung der Konfiguration.
- `README.md` – Übersicht und Dateistruktur dieses Repositories.